0076. Minimum Window Substring

Category: Sliding Window | LeetCode: https://leetcode.com/problems/minimum-w

Problem

- Return the smallest substring of s that contains every character of t (with multiplicity). Return "" if none.
- Example: s = "ADOBECODEBANC", t = "ABC" → "BANC".

Intuition

- Expand the right edge until the window covers t, then shrink from the left as long as it still covers t, recording the best.
- A counter "missing" (characters of t still needed) tells us in O(1) whether the window is valid.

Brute Force

- Check every substring: O(n^2 charset).

Optimal Approach

- Need[c] = count in t; missing = |t|.
- For r: if need[s[r]] > 0, missing--; need[s[r]]--.
- While missing == 0: record window; need[s[l]]++; if need[s[l]] > 0, missing++; l++.

Worked Example

```
s = "ADOBECODEBANC", t = "ABC".

r=5  window ADOBEC covers -> best=ADOBEC(6); shrink drops A -> missing
r=10 window DOBECODEBA covers -> shrink to CODEBA(6), then ODEBA lacks C
r=12 window ODEBANC covers -> shrink to BANC(4) best
answer "BANC"
```

Java Solution

```java
public class MinimumWindowSubstring {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        for (char c : t.toCharArray()) need[c]++;
        int missing = t.length(), l = 0, bestL = 0, bestLen = Integer.MAX_VALUE;
        for (int r = 0; r < s.length(); r++) {
            if (need[s.charAt(r)]-- > 0) missing--;
            while (missing == 0) {
                if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
                if (++need[s.charAt(l++)] > 0) missing++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestL, bestL + bestLen);
    }
}
```

Complexity

- Time: O(|s| + |t|). Space: O(charset).

Pitfalls

- Need[] goes negative for surplus characters; only increment missing when it climbs back above 0.
- Store the best start and length, not substrings, to avoid O(n^2) copying.